BEASTEST ORIGINALIUM / L2

Réconciliation avec le produit logiciel

Partie B - Semaines 4 à 6

BEASTEST - Méthode d'excellence en qualité logicielle

Auteur

Léo Michel Bélanger

ORIGINALIUM

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLSTT~

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLSTT~

Mot d'ouverture

Par Léo Michel Bélanger

Salut,

Tu as fait quelque chose d'important en complétant la Partie A. Tu as osé regarder le produit logiciel en face - pas juste comme un outil qu'on teste, mais comme un objet vivant, complexe, porteur d'intentions et de contraintes. C'est une transformation réelle, et tu devrais en être fier.

Maintenant, il est temps d'aller plus loin. La Partie B, ce sont les semaines 4, 5 et 6, et elles t'amènent dans les profondeurs techniques du produit logiciel. On ne parle plus de définitions générales - on plonge dans ce qui distingue vraiment les types de logiciels entre eux, dans ce que l'architecture impose comme contraintes à ta stratégie de qualité, et dans comment le cycle de vie d'un produit change tout à ce que tu dois surveiller, tester et valider.

Tu vas apprendre à distinguer les exigences de qualité d'un logiciel embarqué de celles d'un SaaS cloud. Tu vas comprendre pourquoi une architecture microservices génère des risques que tu ne verrais jamais dans un monolithe. Et tu vas découvrir comment la dette technique, le refactoring et la gestion des releases font partie intégrante de ton travail de professionnel de la qualité.

ISO 25010 sera ton ancre tout au long de ces semaines. Pas comme une liste à cocher, mais comme un cadre vivant à appliquer avec intelligence et jugement.

Avance avec curiosité. Chaque exercice est une invitation à réfléchir, à noter, à questionner. Tu n'es pas seul dans ce parcours - BEASTEST est une méthode construite pour toi, avec toi, à ton rythme.

C'est parti pour la Partie B.

Léo Michel Bélanger

Auteur, BEASTEST ORIGINALIUM

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

SEMAINE 4 - Les types de produits logiciels et leurs exigences de qualité spécifiques

Apprendre à reconnaître ce que chaque type de logiciel exige de toi en matière de qualité.

Tous les logiciels ne sont pas créés égaux. Un logiciel embarqué dans un défibrillateur cardiaque n'a pas les mêmes exigences de qualité qu'une application mobile de livraison de repas. La semaine 4 t'outille pour distinguer ces univers, adapter ta pensée et appliquer ISO 25010 avec précision selon le contexte du produit.

Objectif de la semaine 4

Être capable de classifier un produit logiciel selon son type, identifier les exigences de qualité qui lui sont propres et amorcer une démarche de test adaptée à son contexte spécifique.

JOUR 1

Classifier les types de logiciels : embarqué, web, mobile, desktop, SaaS, API, microservices

Mise en contexte

Avant de tester un produit logiciel, tu dois savoir ce qu'il est. La classification des logiciels n'est pas un exercice académique : c'est la base de ta stratégie de qualité. Chaque type impose des contraintes différentes sur les outils, les environnements, les données de test, les risques et les critères d'acceptation.

Les 7 grandes catégories à connaître

Type	Définition	Exemples typiques	Contrainte principale
Embarqué	Logiciel intégré à du matériel physique, fonctionnant en autonomie ou avec des ressources limitées	Firmware d'imprimante, logiciel de pacemaker, ECU automobile	Temps réel, fiabilité critique, ressources contraintes
Web	Application accessible via un navigateur, hébergée sur un serveur	Portail bancaire, boutique en ligne, intranet RH	Compatibilité navigateurs, sécurité, performance réseau
Mobile	Application installée sur un appareil mobile (iOS, Android)	Applications de messagerie, GPS, santé	Fragmentation des appareils, connectivité variable, autonomie batterie
Desktop	Application installée localement sur un ordinateur	Suite Office, logiciel de comptabilité, IDE	Compatibilité OS, installation/mise à jour, ressources locales
SaaS	Logiciel fourni comme service en nuage, abonnement multi-locataires	Salesforce, Jira, Slack, GitHub	Disponibilité, multi-tenancy, SLA, mises à jour continues
API	Interface de programmation exposant des fonctionnalités à d'autres systèmes	API REST de paiement, API météo, API de géolocalisation	Contrat d'interface, versioning, sécurité, performance
Microservices	Architecture de services indépendants communiquant	Plateforme e-commerce décomposée en services (catalogue,	Orchestration, résilience, tests d'intégration inter-services

Type	Définition	Exemples typiques	Contrainte principale
	entre eux, déployables séparément	panier, paiement)	

Questions de réflexion - Jour 1

1. Pense à ton contexte professionnel actuel (ou passé). Quel type de logiciel testes-tu le plus souvent? Quelles contraintes spécifiques observes-tu au quotidien?

2. As-tu déjà travaillé sur un projet où le type de logiciel n'était pas clairement défini? Quelles difficultés cela a-t-il créées dans ta démarche de test?

3. Certains produits peuvent combiner plusieurs types (par exemple, une application mobile qui consomme des microservices via une API). Comment classifies-tu ce type de produit hybride pour définir ta stratégie de qualité?

Exercice pratique - Jour 1

Choisis 3 logiciels que tu utilises régulièrement dans ta vie professionnelle ou personnelle. Pour chacun :

- Identifie son type principal selon le tableau ci-dessus.
- Identifie s'il est hybride (plusieurs types combinés).
- Note la contrainte de qualité qui te semble la plus critique pour ce produit.

Documente tes réponses dans les pages de notes à la fin de ce livre.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

JOUR 2

Analyser les exigences de qualité spécifiques à chaque type de logiciel

---

Mise en contexte

Une fois le type de logiciel identifié, la question suivante s'impose : qu'est-ce que la qualité signifie concrètement pour ce produit? Ce n'est pas la même chose de tester la fiabilité d'un logiciel embarqué médical et de valider la performance d'une API de paiement en heure de pointe. La compréhension fine des exigences de qualité par type de produit est ce qui distingue un testeur réactif d'un professionnel de la qualité stratégique.

Matrice des exigences de qualité par type

Type	Qualités prioritaires	Exigences de test associées
Embarqué	Fiabilité, sécurité fonctionnelle, performance temps réel, maintenabilité	Tests de charge matérielle, tests de tolérance aux pannes, tests de durée (soak tests), certification (IEC 61508, DO-178C)
Web	Utilisabilité, compatibilité, sécurité, performance, accessibilité	Tests multi-navigateurs, tests d'intrusion, tests de charge, tests d'accessibilité WCAG
Mobile	Portabilité, performance sur appareils variés, utilisabilité, autonomie	Tests sur appareils réels et émulateurs, tests de consommation de batterie, tests de connectivité intermittente
Desktop	Compatibilité OS, stabilité, installabilité, performance locale	Tests de compatibilité multi-OS, tests d'installation/désinstallation, tests de régression sur versions
SaaS	Disponibilité, scalabilité, sécurité multi-locataires, conformité	Tests de montée en charge, tests d'isolation des données entre clients, tests de SLA, tests de conformité RGPD
API	Fiabilité, performance,	Tests de contrat (contract

Type	Qualités prioritaires	Exigences de test associées
	sécurité, conformité au contrat	testing), tests de charge sur endpoints, tests d'authentification/autorisation
Microservices	Résilience, indépendance, cohérence des données, observabilité	Tests d'intégration entre services, tests de chaos (chaos engineering), tests de circuit breaker

Questions de réflexion - Jour 2

1. Dans ta pratique, as-tu tendance à tester les mêmes dimensions de qualité peu importe le type de logiciel? Qu'est-ce qui t'amènerait à adapter ton approche?

2. Quelle dimension de qualité est, selon toi, la plus souvent négligée dans les projets que tu connais? Pourquoi?

3. Comment les exigences réglementaires (comme RGPD, HIPAA, PCI-DSS) changent-elles le portrait des exigences de qualité pour un logiciel donné?

Exercice pratique - Jour 2

Pour le produit logiciel principal de ton contexte professionnel actuel :

- Liste les 5 exigences de qualité les plus importantes, en ordre de priorité.
- Pour chacune, identifie si elle est couverte par des tests existants, partiellement couverte ou non couverte.
- Note les lacunes identifiées : elles constitueront des points d'amélioration concrets pour ton équipe.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 3

Étude de cas - Différences de test entre un logiciel embarqué et une application web

---

Mise en contexte

Les différences entre le test d'un logiciel embarqué et celui d'une application web sont profondes - elles touchent l'environnement de test, les outils, les profils de risque, les cycles de release et la culture qualité. Cette étude de cas te permet d'ancrer ces différences dans des situations concrètes.

Étude de cas : Deux projets, deux univers

Projet A - Logiciel embarqué : Système de gestion de l'alimentation électrique d'un véhicule électrique. Ce logiciel contrôle la charge de la batterie, la distribution d'énergie aux moteurs et les protections en cas de surtension. Une défaillance peut être catastrophique pour la sécurité physique des occupants.

Projet B - Application web : Plateforme SaaS de gestion des ressources humaines utilisée par 500 entreprises. Elle gère les dossiers des employés, les paies, les congés et les évaluations de performance. Une défaillance cause des pertes financières et une perte de confiance.

Dimension	Projet A - Embarqué	Projet B - Web/SaaS
Environnement de test	Banc de test matériel, simulateurs de véhicule, équipements de mesure spécialisés (oscilloscopes, analyseurs de bus CAN)	Navigateurs multiples, environnements cloud, outils de performance (JMeter, Gatling), environnements de staging
Types de tests prioritaires	Tests de sécurité fonctionnelle, tests de durée (1000h+), tests de tolérance aux pannes, tests de temps réel	Tests fonctionnels, tests de performance, tests de sécurité applicative (OWASP), tests d'accessibilité
Gestion des versions	Cycles longs (6-18 mois),	Cycles courts

Dimension	Projet A - Embarqué	Projet B - Web/SaaS
	certification obligatoire avant release, pas de mise à jour à la volée	(Agile/DevOps), déploiements continus possibles, rollback rapide en cas de problème
Critères d'acceptation	Normes de sécurité (ISO 26262 pour l'automobile), zéro défaut de sécurité critique, certification formelle requise	SLA de disponibilité (99,9%), temps de réponse sous 2 secondes, score WCAG AA minimum
Coût d'un défaut en production	Potentiellement catastrophique : rappel de véhicules, blessures, responsabilité légale majeure	Financier et réputationnel : perte de clients, pénalités de SLA, atteinte à la marque
Automatisation	Difficile et coûteuse à mettre en place, nécessite du matériel spécialisé; souvent combinée à des tests manuels formels	Très mature : pipelines CI/CD, tests UI automatisés (Selenium, Playwright), tests d'API (Postman, RestAssured)

Questions de réflexion - Jour 3

1. Si tu devais passer d'un rôle de testeur web à un rôle de testeur embarqué, quelles compétences devrais-tu acquérir en priorité?

2. Dans quelle mesure les pratiques Agile et DevOps sont-elles applicables au monde du logiciel embarqué? Quels sont les obstacles principaux?

3. Comment la notion de "risque acceptable" diffère-t-elle entre ces deux contextes? Qui décide du niveau de risque acceptable, et selon quels critères?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 4

Identifier les stakeholders et leurs attentes qualité selon le type de produit

Mise en contexte

La qualité n'existe pas dans le vide - elle est toujours définie par rapport aux attentes de quelqu'un. Ces "quelqu'uns" sont tes stakeholders. Selon le type de produit logiciel, les parties prenantes changent radicalement, et leurs attentes en matière de qualité aussi. Comprendre qui tes stakeholders sont - et ce qu'ils entendent par "qualité" - est une compétence fondamentale pour tout professionnel de la qualité logicielle.

Carte des stakeholders par type de produit

Type de produit	Stakeholders clés	Attentes qualité principales
Embarqué	Ingénieurs matériel, organismes de certification, équipes de sécurité, équipes légales, clients finaux (OEM)	Conformité normative, zéro défaut critique, traçabilité complète des tests, fiabilité long terme
Web	Utilisateurs finaux, équipes marketing, équipes UX, propriétaires de produit, équipes de sécurité informatique	Facilité d'utilisation, performance perçue, accessibilité, sécurité des données personnelles
Mobile	Utilisateurs finaux, stores d'applications (Apple, Google), équipes marketing, équipes support	Expérience utilisateur fluide, autonomie batterie, compatibilité appareils, rapidité de démarrage
SaaS	Administrateurs client, responsables IT des organisations clientes, équipes commerciales, partenaires intégrateurs	Disponibilité contractuelle (SLA), isolation des données entre clients, conformité réglementaire, support réactif
API	Développeurs consommateurs de l'API, architectes de solutions, partenaires tiers, équipes sécurité	Stabilité du contrat d'interface, documentation précise, performance, sécurité de l'authentification
Microservices	Équipes de développement des services consommateurs, architectes, équipes d'opérations (SRE/DevOps)	Indépendance des déploiements, résilience, observabilité, performance inter-services

Point de vigilance

Les conflits entre stakeholders sont fréquents. L'équipe commerciale peut vouloir une release rapide alors que l'équipe sécurité identifie des vulnérabilités critiques. Ton rôle en tant que professionnel de la qualité est d'objectiver ces tensions avec des données - pas de les arbitrer politiquement.

Questions de réflexion - Jour 4

1. Dans ton contexte actuel, as-tu une vue claire de tous tes stakeholders et de leurs attentes en matière de qualité? Si non, comment pourrais-tu les identifier de façon systématique?

2. As-tu déjà vécu un conflit entre deux stakeholders aux attentes incompatibles? Comment cela a-t-il été résolu, et quel était ton rôle dans ce processus?

3. Certains stakeholders ont des attentes implicites (non formulées) en matière de qualité. Comment les identifies-tu et comment les fais-tu émerger dans les conversations de projet?

Exercice pratique - Jour 4

Construis une carte de stakeholders pour ton produit logiciel actuel :

- Liste tous les stakeholders que tu peux identifier (internes et externes).
- Pour chacun, note son attente qualité principale (une phrase concrète, pas abstraite).
- Identifie les potentiels conflits d'attentes entre stakeholders.

- Note comment ta stratégie de test actuelle adresse (ou n'adresse pas) chacune de ces attentes.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 5

Cartographier les risques qualité selon le type de produit logiciel

---

Mise en contexte

L'analyse de risque est au coeur d'une stratégie de test efficace. Le risque n'est pas une notion abstraite : c'est la probabilité qu'un problème survienne, multipliée par l'impact de ce problème sur les stakeholders et sur l'organisation. Selon le type de produit, les risques les plus critiques changent radicalement. Cartographier ces risques te permet de concentrer tes efforts là où ils ont le plus de valeur.

Cadre d'analyse des risques qualité

Pour chaque risque, on évalue deux dimensions :

- **Probabilité :** À quelle fréquence ce type de défaut se manifeste-t-il dans ce type de produit?
- **Impact :** Quelle est la gravité des conséquences si ce défaut atteint la production?

Type de produit	Risques qualité les plus critiques	Stratégie de mitigation recommandée
Embarqué	Défaillance en conditions extrêmes (température, vibration), dépassement de délais temps réel, corruption mémoire, mise à jour défaillante du firmware	Tests environnementaux, analyse FMEA, tests de stress prolongés, gestion formelle des exigences (DOORS, ReqIF)

Type de produit	Risques qualité les plus critiques	Stratégie de mitigation recommandée
Web	Injection SQL/XSS, dégradation de performance sous charge, bris de compatibilité navigateur, fuite de données personnelles	Tests de pénétration réguliers, tests de charge automatisés, matrice de compatibilité, audits de conformité RGPD
Mobile	Plantage sur certains modèles d'appareils, consommation excessive de batterie, permissions abusives, comportement hors connexion défaillant	Tests sur farm d'appareils réels, tests de consommation énergétique, revue des permissions, tests mode avion
SaaS	Fuite de données inter-clients (isolation), indisponibilité prolongée, migration de données défaillante, contournement des contrôles d'accès	Tests d'isolation multi-tenant, tests de résilience, tests de migration, tests de contrôle d'accès basé sur les rôles
API	Bris de contrat d'interface sans préavis, déni de service par surcharge, authentification insuffisante, réponses incorrectes en cas d'erreur	Contract testing (Pact), rate limiting, tests de sécurité API (OWASP API Security Top 10), tests de gestion des erreurs
Microservices	Cascade de pannes entre services, incohérence des données distribuées, dérive de configuration, latence excessive inter-services	Chaos engineering, tests de circuit breaker, tests de cohérence éventuelle, observabilité (traces distribuées)

Questions de réflexion - Jour 5

1. Dans ton projet actuel, le profil de risque est-il documenté et partagé avec l'équipe? À quelle fréquence est-il mis à jour?

2. As-tu déjà découvert en production un risque que tu n'avais pas anticipé? Quel signal t'avait manqué lors de la phase de planification?

3. Comment prioriseras-tu tes efforts de test lorsque le temps et les ressources sont limités? Quelle méthodologie utilises-tu pour rationaliser cette décision?

Exercice pratique - Jour 5

Crée une matrice de risques qualité pour ton produit actuel :

- Identifie 5 à 10 risques qualité pertinents pour ton type de produit.
- Évalue chaque risque sur une échelle de 1 à 5 pour la probabilité et pour l'impact.
- Calcule un score de risque (probabilité x impact) et priorise-les en conséquence.
- Pour chaque risque à score élevé (8+), note la stratégie de mitigation que tu appliques ou devrais appliquer.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 6

Appliquer ISO 25010 aux différents types de produits logiciels

---

Mise en contexte

ISO 25010 est le modèle de référence international pour la qualité du produit logiciel. Il définit 8 caractéristiques de qualité principales et leurs sous-caractéristiques. Mais attention : ISO 25010 n'est pas un formulaire à remplir case par case. C'est un cadre à interpréter intelligemment selon le type de produit que tu as entre les mains. Certaines caractéristiques sont primordiales pour un type de produit et secondaires pour un autre.

Les 8 caractéristiques de qualité ISO 25010

Caractéristique	Définition synthétique
Adéquation fonctionnelle	Le logiciel fait ce pour quoi il a été conçu, complètement et correctement.
Efficacité de la performance	Le logiciel utilise efficacement les ressources pour atteindre sa performance dans les conditions définies.
Compatibilité	Le logiciel fonctionne correctement en coexistence avec d'autres produits et peut échanger des données avec eux.
Utilisabilité	Les utilisateurs cibles peuvent atteindre leurs objectifs avec efficacité, efficience et satisfaction.
Fiabilité	Le logiciel exécute ses fonctions de façon stable et sans défaillance dans des conditions normales et anormales.
Sécurité	Le logiciel protège les données et les systèmes contre les accès non autorisés, tout en garantissant la disponibilité aux utilisateurs légitimes.
Maintenabilité	Le logiciel peut être modifié efficacement pour corriger des défauts, améliorer ses fonctions ou s'adapter à de nouveaux environnements.
Portabilité	Le logiciel peut être transféré efficacement d'un environnement à un autre.

ISO 25010 appliqué par type de produit - Priorisation suggérée

Type	Priorité 1	Priorité 2	Priorité 3	Souvent sous-évaluée
Embarqué	Fiabilité	Performance	Sécurité	Maintenabilité
Web	Sécurité	Utilisabilité	Performance	Compatibilité
Mobile	Utilisabilité	Portabilité	Performance	Fiabilité (offline)
SaaS	Fiabilité	Sécurité	Performance	Maintenabilité
API	Adéquation fonctionnelle	Sécurité	Performance	Compatibilité
Microservices	Fiabilité	Maintenabilité	Performance	Portabilité

Questions de réflexion - Jour 6

1. Si tu devais expliquer ISO 25010 à un chef de projet non technique, comment le ferais-tu en moins de 3 minutes? Pratique cet exercice de vulgarisation.

2. As-tu déjà vu un projet échouer à cause d'une caractéristique de qualité négligée qui aurait été couverte par ISO 25010? Laquelle?

3. Comment intégrerais-tu ISO 25010 dans ton processus de définition des critères d'acceptation, sans alourdir inutilement la démarche?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 7

Créer une fiche de référence personnalisée par type de produit logiciel

---

Mise en contexte

Tu as maintenant les éléments pour synthétiser la semaine 4 en quelque chose de concret et réutilisable : ta fiche de référence personnalisée. Ce livrable est à toi - il reflète ton contexte, ta réalité professionnelle et ta compréhension. Il deviendra un outil de travail que tu amélioreras tout au long de ta carrière.

Structure d'une fiche de référence par type de produit

Pour chaque type de logiciel pertinent dans ton contexte, ta fiche devrait contenir :

- **Identification du type :** Nom, définition courte, exemples dans ton secteur.
- **Stakeholders principaux :** Qui sont-ils, qu'attendent-ils?
- **Top 5 des exigences de qualité :** En ordre de priorité pour ce type.
- **Top 3 des risques qualité :** Les plus fréquents et les plus impactants.

- **Caractéristiques ISO 25010 prioritaires :** Avec justification.
- **Types de tests indispensables :** Avec outils et approches recommandés.
- **Notes personnelles :** Lessons apprises, pièges à éviter, ressources utiles.

Exercice pratique - Jour 7

Crée ta fiche de référence pour les 2 ou 3 types de logiciels les plus pertinents dans ton contexte professionnel. Utilise le canevas ci-dessus comme structure. Tu peux utiliser les pages de notes à la fin de ce livre, ou créer un document numérique que tu conserveras dans ton portfolio professionnel.

Conseil : Ne cherche pas la perfection au premier jet. Lance-toi, et reviens enrichir ta fiche au fil de tes expériences. Une fiche vivante vaut mieux qu'une fiche parfaite jamais commencée.

Questions de réflexion - Jour 7

1. Quelle information de la semaine 4 t'a le plus surpris ou remis en question une croyance que tu avais sur ton domaine?

2. Comment vas-tu intégrer cette fiche de référence dans ton flux de travail quotidien? Quel format (numérique, papier, wiki d'équipe) est le plus adapté à ta réalité?

3. Si tu partageais cette fiche avec ton équipe, quelles conversations cela pourrait-il déclencher? Est-ce que tu te sens prêt à initier ces conversations?

TTLST~ - Exercice de Totalisation - Semaine 4

Dimension 1 - Substantif linguistique : "Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser."

Dimension 2 - Philosophie ORIGINALIUM : "Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption."

Exercice de totalisation : Cette semaine, tu as exploré sept types de produits logiciels, chacun avec ses exigences propres, ses stakeholders, ses risques et ses critères de qualité spécifiques. À première vue, ils semblent très différents - et ils le sont. Mais ils partagent tous un objectif fondamental : créer de la valeur pour des humains, de façon fiable, sécurisée et utilisable.

Dans tes notes, réponds à ces questions de totalisation :

- Quels fils conducteurs communs traversent tous les types de produits logiciels lorsqu'on parle de qualité? (Indice : pense à ISO 25010 comme cadre unificateur.)
- Comment ta vision de la qualité logicielle a-t-elle évolué cette semaine? Qu'est-ce qui a changé dans ta façon de penser le produit que tu testes?
- Si tu devais expliquer à quelqu'un la différence entre "tester un logiciel" et "assurer la qualité d'un produit logiciel", qu'est-ce que cette semaine t'aurait appris de nouveau à dire?
- Comment tous ces types de logiciels coexistent-ils dans un écosystème numérique réel? Dessine mentalement (ou sur papier) les interactions entre eux dans un produit que tu connais bien.

La totalisation, c'est refuser de voir les parties séparément quand l'ensemble révèle quelque chose que chaque partie ne peut pas montrer seule.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

SEMAINE 5 - Architecture logicielle et impact sur la qualité

Comprendre comment les décisions architecturales définissent ta stratégie de test, avant même qu'une ligne de code soit écrite.

L'architecture logicielle n'est pas le territoire exclusif des architectes et des développeurs seniors. En tant que professionnel de la qualité, l'architecture est l'une de tes principales sources d'information pour concevoir une stratégie de test pertinente. Elle te dit où se trouvent les joints d'assemblage fragiles, les dépendances critiques et les points de défaillance potentiels. La semaine 5 t'emmène dans cet univers architectural avec un objectif clair : comprendre assez pour tester intelligemment.

Objectif de la semaine 5

Être capable de lire une architecture logicielle avec les yeux d'un professionnel de la qualité, d'identifier les défis de test spécifiques à chaque paradigme architectural et de proposer une stratégie de test adaptée.

JOUR 1

Architecture monolithique vs microservices - Impacts sur les tests

---

Mise en contexte

Le choix entre une architecture monolithique et une architecture microservices est l'une des décisions les plus structurantes d'un projet logiciel. Et ses conséquences sur la qualité et les tests sont profondes. Chaque paradigme a ses forces, ses faiblesses et ses défis de test propres. Les comprendre, c'est éviter de plaquer une stratégie de test inadaptée sur une architecture qui l'invalide.

Architecture monolithique

Dans un monolithe, toutes les fonctionnalités de l'application sont regroupées dans un seul déployable. La base de code est unifiée, la communication entre modules est directe (appels de fonctions en mémoire) et il n'y a qu'une seule base de données typiquement.

Avantages pour les tests	Défis pour les tests
Tests d'intégration simples (tout est dans le même processus)	Le monolithe grossit avec le temps, rendant les tests de régression de plus en plus longs et coûteux
Un seul environnement de déploiement à gérer	Un défaut dans un module peut affecter tout le système - les impacts sont difficiles à isoler
Débogage facilité (stack trace unifiée)	Les releases impliquent de retester l'ensemble, même pour un changement mineur
Moins de complexité de configuration dans les environnements de test	La vitesse d'exécution des tests diminue à mesure que la base de code grandit

Architecture microservices

Dans une architecture microservices, l'application est décomposée en services indépendants, chacun responsable d'une capacité métier spécifique, déployable séparément et communiquant via des interfaces réseau (APIs, messages).

Avantages pour les tests	Défis pour les tests
Chaque service peut être testé de façon indépendante (isolation naturelle)	Tests d'intégration complexes : les interactions entre services sont nombreuses et difficiles à reproduire
Les releases sont plus ciblées - on ne reteste que ce qui a changé	La complexité des environnements de test explose (orchestration de conteneurs, services simulés)
La résilience peut être testée de façon granulaire (circuit breaker par service)	Le débogage distribué est difficile sans observabilité (logs, traces, métriques)
Les contrats d'interface entre services peuvent être validés automatiquement	Les tests end-to-end deviennent complexes et fragiles à maintenir

Questions de réflexion - Jour 1 (Semaine 5)

1. Ton produit actuel est-il plus proche d'un monolithe ou d'une architecture microservices? Comment cela influence-t-il concrètement ton travail de test quotidien?

2. De nombreuses organisations sont en transition du monolithe vers les microservices. Quels risques qualité spécifiques cette période de transition génère-t-elle, et comment les adresser?

3. L'architecture microservices exige une sophistication de l'outillage de test beaucoup plus grande. Comment une équipe à maturité de test limitée devrait-elle aborder ce défi?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 2

Architecture en couches - Stratégies de test par couche

Mise en contexte

L'architecture en couches (layered architecture) est l'un des patterns architecturaux les plus répandus. Elle organise le logiciel en couches distinctes - généralement présentation, logique métier et accès aux données - chacune avec une responsabilité bien définie. Cette séparation est une opportunité pour la qualité : elle permet de cibler chaque couche avec des stratégies de test adaptées.

Les trois couches et leurs stratégies de test

Couche	Responsabilité	Types de tests appropriés	Outils typiques
Présentation	Interface utilisateur,	Tests UI	Selenium,

Couche	Responsabilité	Types de tests appropriés	Outils typiques
(UI)	rendu visuel, navigation, interactions utilisateur	automatisés, tests d'accessibilité, tests d'utilisabilité, tests de compatibilité navigateur	Playwright, Cypress, Appium, axe (accessibilité)
Logique métier (Business Layer)	Règles de gestion, calculs, workflows, validations métier	Tests unitaires, tests d'intégration fonctionnelle, tests de règles métier, tests de cas limites	JUnit, NUnit, pytest, Cucumber/Gherkin, FitNesse
Accès aux données (Data Layer)	Persistance des données, requêtes, transactions, intégrité référentielle	Tests de base de données, tests de migration, tests de performance des requêtes, tests de cohérence des données	DbUnit, Testcontainers, SQL profilers, scripts de validation de migration

Le principe de la pyramide de test

Dans une architecture en couches, la pyramide de test de Mike Cohn s'applique naturellement : beaucoup de tests unitaires (couche métier), moins de tests d'intégration et encore moins de tests UI. Cette répartition maximise la couverture tout en minimisant le coût de maintenance des tests.

Questions de réflexion - Jour 2 (Semaine 5)

1. Dans ton projet actuel, quelle couche est la moins bien couverte par les tests? Quelles en sont les raisons (technique, humaine, organisationnelle)?

2. Comment les modifications apportées à la couche de données peuvent-elles silencieusement briser des fonctionnalités dans la couche de présentation? Quels mécanismes de test permettent de détecter ces régressions?

3. Certaines architectures modernes (comme les applications React avec API GraphQL) brouillent les frontières entre les couches. Comment adaptes-tu ta

stratégie de test dans ce cas?

Exercice pratique - Jour 2 (Semaine 5)

Pour ton projet actuel, réalise un audit de couverture par couche :

- Estime le pourcentage de couverture de test (automatisée ou manuelle) dans chaque couche.
- Identifie les zones à risque dans chaque couche.
- Propose un plan d'amélioration ciblé sur la couche la plus vulnérable.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 3

Architecture orientée événements - Défis de qualité et approches de test

---

Mise en contexte

L'architecture orientée événements (Event-Driven Architecture - EDA) repose sur la production, la détection et la réaction à des événements. Des composants (producteurs) émettent des événements ; d'autres (consommateurs) y réagissent de façon asynchrone. Ce paradigme apporte une grande flexibilité et scalabilité, mais il génère des défis de qualité uniques qui doivent être anticipés dans ta stratégie de test.

Défis de qualité spécifiques à l'EDA

Défi	Description	Approche de test recommandée
Asynchronisme	Les réponses ne sont pas immédiates; les tests synchrones traditionnels ne fonctionnent pas directement	Tests avec mécanismes de polling ou de wait, assertions avec timeout, utilisation de frameworks asynchrones (awaitility)
Ordre des événements	L'ordre de traitement des événements peut varier, causant des incohérences d'état	Tests de séquencement, tests de réordering, validation de l'idempotence des handlers
Perte d'événements	Un événement peut être perdu en cas de panne du broker ou du consommateur	Tests de résilience du broker (Kafka, RabbitMQ), tests de replay d'événements, validation des politiques de retry
Doublons d'événements	En cas de retry, un même événement peut être traité plusieurs fois	Tests d'idempotence : vérifier que traiter un événement N fois produit le même résultat que le traiter une fois
Cohérence éventuelle	L'état du système peut être temporairement incohérent entre plusieurs consommateurs	Tests de convergence : valider que le système atteint l'état cohérent dans un délai acceptable
Schema drift	Le format d'un événement peut changer avec le temps, brisant les consommateurs qui n'ont pas été mis à jour	Tests de compatibilité de schéma (Schema Registry), tests de rétrocompatibilité, contract testing sur les événements

Questions de réflexion - Jour 3 (Semaine 5)

1. Comment testerais-tu un flux de traitement de commandes e-commerce basé sur des événements (commande créée, paiement traité, expédition déclenchée)? Quels scénarios de test seraient critiques?

2. Dans une architecture EDA, comment sais-tu qu'un test a "échoué" si le système est conçu pour être éventuellement cohérent? Où mets-tu la frontière entre un comportement acceptable et un défaut?

3. Comment l'observabilité (logs structurés, traces distribuées, métriques) devient-elle une partie intégrante de ta stratégie de test dans un contexte

événementiel?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 4

API REST et GraphQL - Tests de contrat, de performance et de sécurité

Mise en contexte

Les APIs sont les artères du logiciel moderne. Qu'il s'agisse d'une API REST ou GraphQL, elles exposent des capacités à d'autres systèmes et doivent être testées avec rigueur sur trois dimensions essentielles : le contrat (est-ce que l'API fait exactement ce qu'elle promet?), la performance (est-ce qu'elle le fait dans des délais acceptables?) et la sécurité (est-ce qu'elle le fait sans exposer de vulnérabilités?).

Comparatif REST vs GraphQL pour les tests

Dimension	API REST	API GraphQL
Structure	Endpoints multiples et fixes (ex: /users, /orders)	Endpoint unique avec requêtes flexibles définies par le client
Tests de contrat	Spécifications OpenAPI/Swagger, Pact pour le consumer-driven contract testing	Schéma GraphQL (SDL), tests de conformité au schéma, introspection
Tests de performance	Tests de charge sur des endpoints spécifiques (JMeter, Gatling, k6)	Risque de requêtes N+1, requêtes profondément imbriquées - nécessite des tests de complexity et de depth limiting
Tests de sécurité	OWASP API Security Top 10 : Broken Object Level Authorization, Excessive Data Exposure, Lack of Rate	Introspection malveillante, batching attacks, requêtes excessivement coûteuses, injection via variables

Dimension	API REST	API GraphQL
	Limiting...	
Outils spécifiques	Postman, RestAssured, Newman, Insomnia, Karate	GraphQL Playground, Apollo Studio, graphql-tag, whatsup.graphql

Les 3 niveaux de tests d'API

- **Tests de contrat :** Valider que l'API respecte son contrat d'interface (codes de statut corrects, structure de réponse conforme, comportements documentés pour les cas d'erreur).
- **Tests de performance :** Mesurer les temps de réponse sous différents niveaux de charge, identifier les goulets d'étranglement, valider les SLA de performance.
- **Tests de sécurité :** Valider l'authentification (OAuth 2.0, JWT, API keys), l'autorisation (qui peut faire quoi), la protection contre les injections et la gestion sécurisée des données sensibles.

Questions de réflexion - Jour 4 (Semaine 5)

1. Dans ton contexte, les APIs sont-elles documentées de façon suffisamment précise pour permettre la création de tests de contrat automatisés? Sinon, comment améliorerais-tu cette situation?

2. Comment testes-tu actuellement la sécurité des APIs dans tes projets? Qu'est-ce qui manque dans ton approche actuelle?

3. Quelle est la différence entre un test de performance d'API et un test de charge d'application web? Dans quels cas chacun est-il plus approprié?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 5

Architecture cloud-native - Tests de résilience, chaos engineering et scalabilité

Mise en contexte

Une application cloud-native est conçue dès le départ pour exploiter les capacités du nuage : scalabilité élastique, distribution géographique, tolérance aux pannes, déploiement continu. Ces propriétés changent fondamentalement la façon dont tu dois penser et concevoir tes tests. Dans un contexte cloud-native, la question n'est plus "est-ce que le système fonctionne?" mais "est-ce que le système continue de fonctionner - et de façon acceptable - même quand certaines parties tombent en panne?"

Principes de test dans un contexte cloud-native

Principe	Description	Approche de test
Tests de résilience	Valider que le système maintient un niveau de service acceptable en cas de panne partielle	Simulation de pannes (kill pod, réseau dégradé), validation du comportement des circuit breakers, tests de fallback
Chaos Engineering	Introduction contrôlée de perturbations en production ou en staging pour identifier les faiblesses	Chaos Monkey (Netflix), Chaos Toolkit, Litmus Chaos pour Kubernetes; définition d'hypothèses, injection de chaos, observation et correction
Tests de scalabilité	Valider que le système s'adapte à la charge en ajoutant des ressources (scale-out) et en les libérant (scale-in) sans défaillance	Tests de montée en charge progressive, validation de l'auto-scaling, tests de scale-down sous charge légère
Tests de déploiement	Valider les stratégies de déploiement zero-downtime (blue-green, canary, rolling)	Tests de basculement, validation du comportement pendant la transition, tests de rollback automatique
Tests multi-région	Valider le comportement en cas de panne d'une région géographique complète	Simulation de failover géographique, validation de la latence entre régions, cohérence des données distribuées

Point de vigilance

Le chaos engineering ne s'improvise pas. Il faut des garde-fous solides : définir une hypothèse claire avant chaque expérience, avoir des mécanismes d'arrêt d'urgence, commencer en environnement de staging avant d'aller en production, et avoir l'accord explicite des parties prenantes. Un chaos non contrôlé est simplement... du chaos.

Questions de réflexion - Jour 5 (Semaine 5)

1. Ton organisation est-elle prête culturellement et techniquement pour le chaos engineering? Quels seraient les premiers obstacles à surmonter?

2. Comment distingues-tu un test de résilience (planifié, contrôlé) d'un incident de production (subi, non contrôlé)? Qu'est-ce qu'ils ont en commun, et en quoi diffèrent-ils fondamentalement?

3. Quels métriques utilises-tu (ou utiliserais-tu) pour définir le "niveau de service acceptable" lors de tests de résilience? Comment ces métriques sont-elles définies et validées avec les stakeholders?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 6

Infrastructure as Code (IaC) - Tests d'infrastructure et validation de configuration

---

Mise en contexte

L'Infrastructure as Code (IaC) est la pratique de gérer et provisionner l'infrastructure informatique (serveurs, réseaux, bases de données, configurations) via du code versionné, plutôt que via des processus manuels ou des interfaces graphiques. Terraform, Ansible, Pulumi, AWS CloudFormation - ces outils ont transformé la façon dont les environnements sont créés et gérés. Et cette transformation a une implication directe pour toi : l'infrastructure est maintenant du code, et le code doit être testé.

Pourquoi tester l'infrastructure as code?

- Une configuration d'infrastructure incorrecte peut rendre ton application inaccessible, peu performante ou vulnérable, même si ton code applicatif est parfait.
- Les environnements non testés dérivent avec le temps (configuration drift), causant des différences entre staging et production.
- Les changements d'infrastructure sans test peuvent provoquer des pannes en production lors des déploiements.

Niveaux de test pour l'IaC

Niveau	Objectif	Outils	Exemple
Analyse statique	Valider la syntaxe et les meilleures pratiques de sécurité sans déployer	tflint, checkov, tfsec, cfn-nag	Détection de ports ouverts non autorisés, ressources sans chiffrement, credentials hardcodés
Tests unitaires IaC	Valider le comportement des modules d'infrastructure de façon isolée	Terratest (Go), pytest-terraform, localstack	Vérifier qu'un module de réseau crée bien les sous-réseaux avec les bons CIDR
Tests d'intégration	Valider que les composants d'infrastructure fonctionnent correctement ensemble dans un environnement réel	Terratest, Molecule (Ansible), InSpec	Vérifier que l'application peut se connecter à la base de données provisionnée par Terraform

Niveau	Objectif	Outils	Exemple
Validation de configuration	Comparer l'état réel de l'infrastructure à son état désiré pour détecter la dérive	InSpec, ServerSpec, Goss, AWS Config	Vérifier que tous les ports fermés le sont bien, que les versions de logiciels correspondent aux attendus

Questions de réflexion - Jour 6 (Semaine 5)

1. Est-ce que l'infrastructure de tes projets actuels est gérée as code? Si non, quels sont les risques qualité associés à la gestion manuelle de l'infrastructure?

2. Comment définis-tu les critères d'acceptation pour un test d'infrastructure? Qu'est-ce que "une infrastructure qui passe ses tests" signifie concrètement?

3. Quel est le niveau de collaboration entre ton équipe de test et les équipes DevOps/SRE dans tes projets? Comment l'améliorer?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 7

Créer une matrice architecture vs stratégie de test pour ton contexte

---

Mise en contexte

La semaine 5 t'a exposé à cinq paradigmes architecturaux majeurs, chacun avec ses défis de test propres. Il est maintenant temps de faire la synthèse en créant un outil pratique pour ton usage quotidien : ta matrice architecture vs stratégie de test. Cet outil te permettra de répondre rapidement à la question : "On vient de me présenter cette architecture - quels sont les tests indispensables que je dois planifier?"

Canevas de matrice architecture vs stratégie de test

Paradigme architectural	Tests indispensables	Tests spécifiques (ne pas oublier)	Niveau de complexité de l'environnement de test
Monolithique	Tests unitaires, tests d'intégration, tests de régression complète	Tests de performance globale, tests de volume (impact sur temps de réponse)	Faible
Microservices	Tests unitaires par service, contract testing, tests d'intégration inter-services	Chaos engineering, tests de circuit breaker, tests de déploiement indépendant	Élevée
En couches	Tests unitaires (couche métier), tests d'intégration par couche, tests UI	Tests de migration de base de données, tests de compatibilité de schéma	Modérée
Orientée événements	Tests d'idempotence, tests de séquencement, tests de résilience du broker	Tests de schema drift, tests de convergence, tests de replay	Élevée
Cloud-native	Tests de résilience, tests de scalabilité, tests de déploiement	Chaos engineering, tests multi-région, tests d'auto-scaling	Très élevée
IaC	Analyse statique (linting), tests d'intégration d'infrastructure	Validation de configuration (drift detection), tests de sécurité de configuration	Modérée à élevée

Exercice pratique - Jour 7 (Semaine 5)

Adapte ce canevas à ton contexte spécifique :

- Identifie les paradigmes architecturaux présents dans ton environnement professionnel actuel.
- Pour chacun, complète la matrice avec des tests spécifiques à ton produit

(pas des génériques).

- Note les lacunes : quels tests devraient exister mais n'existent pas encore?
- Priorise les 3 améliorations les plus impactantes que tu pourrais proposer à ton équipe.

Questions de réflexion - Jour 7 (Semaine 5)

1. Si tu pouvais recommander une seule amélioration architecturale pour faciliter la testabilité de ton produit actuel, laquelle serait-ce? Comment la justifierais-tu auprès de ton équipe?

2. La complexité de l'environnement de test augmente avec la sophistication architecturale. Comment maintenir l'agilité de l'équipe face à cette complexité croissante?

3. Quels apprentissages de la semaine 5 changes concrètement quelque chose dans ta façon d'aborder ton travail de test dès demain?

TTLST~ - Exercice de Totalisation - Semaine 5

Dimension 1 - Substantif linguistique : "Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser."

Dimension 2 - Philosophie ORIGINALIUM : "Rassembler et réaliser totalement dans son ensemble - avant, pendant et après - en continu, sans interruption."

Exercice de totalisation : L'architecture du produit logiciel n'est pas seulement une question de code - c'est une décision de qualité. Chaque choix architectural crée des opportunités de test et des risques à gérer. La

totalisation de cette semaine t'invite à voir l'architecture non pas comme un fait établi que tu subis, mais comme un territoire que tu dois explorer avec les yeux de la qualité.

Dans tes notes, réponds à ces questions de totalisation :

- **Avant (conception) :** Comment la qualité devrait-elle influencer les décisions architecturales dès la phase de conception? Quelles questions un professionnel de la qualité devrait-il poser lors des revues d'architecture?
- **Pendant (développement) :** Comment la stratégie de test évolue-t-elle au fur et à mesure que l'architecture se précise et que le code prend forme? Comment maintenir une couverture de test cohérente malgré les changements architecturaux en cours de projet?
- **Après (déploiement) :** Comment surveilles-tu la qualité architecturale en production? Quels signaux t'indiquent qu'une décision architecturale passée commence à créer des problèmes de qualité aujourd'hui?
- **En continu :** Qu'est-ce que la qualité architecturale continue signifie pour toi? Comment la maintiens-tu dans le temps, face à la pression des nouvelles fonctionnalités et de la dette technique?

Totaliser l'architecture et la qualité, c'est reconnaître qu'on ne peut pas tester un mauvais design en un produit excellent - et qu'on ne peut pas concevoir une excellente architecture sans penser à comment on la validera.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

Comprendre que la qualité d'un produit logiciel n'est pas un état - c'est un mouvement continuel d'adaptation à travers le temps.

Un produit logiciel nait, croit, mature et décline. À chaque phase de sa vie, les exigences de qualité changent, les risques évoluent, et les stratégies de test doivent s'adapter. La semaine 6 t'outille pour penser la qualité dans la durée - pas seulement pour la prochaine release, mais pour l'ensemble du cycle de vie du produit. C'est une perspective qui te distingue comme professionnel de la qualité stratégique.

Objectif de la semaine 6

Être capable d'adapter ta stratégie de qualité selon la phase du cycle de vie du produit, de gérer proactivement la dette technique et d'anticiper les enjeux de qualité de fin de vie d'un logiciel.

JOUR 1

Phases du cycle de vie produit et adaptation de la qualité

Mise en contexte

Le cycle de vie d'un produit logiciel suit un arc prévisible : introduction (lancement), croissance (adoption), maturité (stabilisation) et déclin (fin de vie). Chaque phase impose des priorités différentes en matière de qualité. Un professionnel de la qualité qui applique la même stratégie à travers toutes ces phases travaille à contre-courant des besoins réels du produit et des stakeholders.

Qualité selon les phases du cycle de vie

Phase	Caractéristiques du produit	Priorités qualité	Types de tests à prioriser
Introduction (lancement)	MVP, base d'utilisateurs limitée, code récent, évolution rapide des fonctionnalités	Adéquation fonctionnelle, utilisabilité de base, stabilité suffisante pour le lancement	Tests fonctionnels des parcours critiques, tests exploratoires, tests d'acceptation utilisateur (UAT)
Croissance (adoption)	Base d'utilisateurs en forte croissance, nouvelles fonctionnalités fréquentes, pression sur la performance	Performance, scalabilité, fiabilité, sécurité renforcée	Tests de charge et de performance, tests de sécurité renforcés, tests de régression automatisés
Maturité (stabilisation)	Base d'utilisateurs stable, évolutions incrémentales, dette technique accumulée, forte exigence de stabilité	Fiabilité, maintenabilité, gestion de la dette technique, rétrocompatibilité	Tests de régression exhaustifs, tests de maintenabilité (code coverage, analyse statique), tests de rétrocompatibilité
Déclin (fin de vie)	Base d'utilisateurs en diminution, maintenance minimale, migration vers successeur	Stabilité, migration sécurisée des données, rétrocompatibilité pendant la transition	Tests de migration, tests de rétrocompatibilité, tests de coexistence avec le produit successeur

Questions de réflexion - Jour 1 (Semaine 6)

1. Dans quelle phase du cycle de vie se trouve ton produit actuel? Comment cela se manifeste-t-il dans ton travail quotidien?

2. As-tu déjà vu une équipe appliquer une stratégie de test de la phase "Introduction" sur un produit en pleine "Maturité" (ou vice versa)? Quelles en étaient les conséquences?

3. Comment la composition et les compétences d'une équipe de qualité devraient-elles évoluer à travers le cycle de vie d'un produit?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 2

Gestion de la dette technique - Identification, priorisation et stratégies de remboursement

---

Mise en contexte

La dette technique est l'accumulation de choix de conception ou d'implémentation sous-optimaux qui facilitent le court terme au détriment du long terme. Comme une dette financière, elle accumule des "intérêts" - sous forme de lenteur de développement, de coûts de maintenance croissants, de risques de défaillance et de difficulté croissante à maintenir la qualité. En tant que professionnel de la qualité, tu es souvent le premier à observer les symptômes de la dette technique à travers les tests.

Symptômes de la dette technique visibles dans les tests

- Les tests de régression prennent de plus en plus de temps et deviennent de plus en plus fragiles.
- Le taux de faux positifs dans les tests automatisés augmente (tests qui échouent sans raison apparente).
- Chaque modification génère des effets de bord inattendus dans des zones non liées.
- La couverture de code automatique stagne malgré les efforts de l'équipe.
- Les nouvelles fonctionnalités nécessitent des efforts de test disproportionnés par rapport à leur complexité.
- Les environnements de test sont difficiles à maintenir stables et reproductibles.

Stratégies de priorisation de la dette technique

Critère	Description	Impact sur la qualité
Risque d'impact	Prioriser la dette dans les zones les plus critiques pour les utilisateurs	Réduction directe des défauts à fort impact
Fréquence de modification	Prioriser la dette dans les zones qui changent le plus souvent	Réduction de la friction quotidienne et accélération des livraisons
Coût de correction croissant	Traiter en priorité la dette dont le coût de correction augmente avec le temps	Éviter que la dette ne devienne un obstacle structurel
Opportunisme	Corriger la dette lorsqu'on touche de toute façon à une zone (approche "boy scout")	Amélioration continue sans effort dédié majeur

Questions de réflexion - Jour 2 (Semaine 6)

1. Comment mesures-tu actuellement la dette technique dans tes projets? Quelles métriques (couverture de code, complexité cyclomatique, nombre de violations de règles de code) utilises-tu?

2. As-tu déjà eu à plaider pour le "remboursement" de dette technique auprès d'un chef de projet ou d'un product owner? Comment as-tu formulé l'argument commercial (business case)?

3. Quelle est la différence entre de la dette technique "intentionnelle" (un compromis conscient et documenté) et de la dette technique "accidentelle" (un mauvais choix non reconnu)? Comment traites-tu chacune différemment?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 3

Refactoring et qualité - Améliorer sans régresser

Mise en contexte

Le refactoring est la transformation du code existant pour améliorer sa structure interne sans modifier son comportement externe. C'est l'outil principal de remboursement de la dette technique. Mais le refactoring sans filet de sécurité est une source majeure de régression. En tant que professionnel de la qualité, tu joues un rôle essentiel dans la mise en place de ce filet et dans la validation que le refactoring a atteint son objectif sans introduire de nouveaux défauts.

Le cycle sécurisé du refactoring

1. **Établir le filet de sécurité :** Avant de refactoriser, s'assurer qu'une couverture de tests suffisante existe pour détecter toute régression. Si la couverture est insuffisante, ajouter des tests d'abord.
2. **Définir l'objectif :** Quel problème de qualité de code le refactoring doit-il résoudre? (Complexité excessive, duplication, couplage fort, manque de lisibilité?)
3. **Refactoriser par petites étapes :** Chaque étape doit être validée par les tests avant de passer à la suivante. Éviter les refactorings massifs non décomposés.
4. **Vérifier l'invariance comportementale :** Tous les tests existants doivent continuer de passer. Tout test en échec indique une régression potentielle.
5. **Mesurer l'amélioration :** Valider que l'objectif du refactoring a été atteint (réduction de la complexité cyclomatique, augmentation de la couverture, suppression des duplications).
6. **Documenter :** Documenter le refactoring effectué, son objectif et les métriques d'amélioration pour le suivi de la dette technique.

Métriques de qualité à surveiller lors du refactoring

Métrique	Définition	Objectif après refactoring
Complexité cyclomatique	Nombre de chemins d'exécution indépendants	Réduction (idéalement sous 10 par méthode/fonction)

Métrique	Définition	Objectif après refactoring
	dans le code	
Couverture de tests	Pourcentage du code couvert par des tests automatisés	Maintien ou augmentation après refactoring
Duplication de code	Pourcentage de code dupliqué (copier-coller)	Réduction significative
Couplage	Degré d'interdépendance entre modules	Réduction du couplage afférent et efférent
Cohésion	Degré auquel les éléments d'un module appartiennent ensemble	Augmentation de la cohésion

Questions de réflexion - Jour 3 (Semaine 6)

1. As-tu déjà vécu un refactoring qui a introduit des régressions en production? Qu'est-ce qui aurait pu l'empêcher?

2. Comment distingues-tu un refactoring qui améliore la qualité d'un refactoring qui ne fait que déplacer la complexité ailleurs? Quels indicateurs objectifs utilises-tu?

3. Dans un contexte Agile avec des sprints courts, comment intègres-tu le refactoring dans le planning sans le voir systématiquement sacrifié au profit des nouvelles fonctionnalités?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 4

Versioning et gestion des releases - Stratégies de test par version

---

Mise en contexte

La gestion des versions (versioning) est la pratique d'attribuer des identifiants uniques aux états successifs d'un logiciel. Elle est intimement liée à la gestion des releases - le processus de livraison de versions du logiciel aux utilisateurs. Pour le professionnel de la qualité, le versioning n'est pas qu'un détail technique : il définit le périmètre de tes tests, les critères de régression et les obligations de rétrocompatibilité.

Semantic Versioning (SemVer) et implications pour les tests

Le Semantic Versioning (SemVer) définit le format **MAJEUR.MINEUR.CORRECTIF** :

Type de version	Signification	Implications pour les tests
MAJEUR (ex: 2.0.0)	Changements incompatibles avec les versions précédentes (breaking changes)	Tests de rétrocompatibilité complets, tests de migration depuis la version précédente, régression exhaustive, tests d'acceptation des nouveaux comportements
MINEUR (ex: 1.3.0)	Nouvelles fonctionnalités rétrocompatibles	Tests des nouvelles fonctionnalités, tests de régression sur les fonctionnalités existantes, validation de la rétrocompatibilité
CORRECTIF (ex: 1.2.5)	Corrections de bugs rétrocompatibles	Tests de régression ciblés sur la zone corrigée, validation que le bug est bien corrigé, vérification qu'aucune régression n'a été introduite

Stratégies de release et tests associés

Stratégie	Description	Tests spécifiques requis
Release classique (planifiée)	Release à date fixe avec périmètre défini à l'avance	Plan de test complet, tests de régression exhaustifs, tests de performance, UAT formel
Continuous Delivery	Chaque changement validé est potentiellement releasable	Pipeline de tests automatisés complet, gates de qualité automatiques,

Stratégie	Description	Tests spécifiques requis
		tests de smoke en production
Feature flags (toggles)	Fonctionnalités déployées mais activées progressivement	Tests de toutes les combinaisons de flags critiques, tests d'activation et de désactivation en production
Canary release	Déploiement progressif à un sous-ensemble d'utilisateurs	Monitoring en production (métriques, erreurs, performance), critères d'arrêt automatique définis

Questions de réflexion - Jour 4 (Semaine 6)

1. Ta stratégie de test est-elle actuellement adaptée à la stratégie de release de ton produit? Quels ajustements seraient nécessaires?

2. Comment gères-tu les tests de régression lorsque plusieurs versions du produit sont supportées simultanément?

3. Quels critères objectifs utilises-tu pour décider si une version est "prête pour la release"? Ces critères sont-ils documentés, connus et acceptés par tous les stakeholders?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 5

Dépréciation de fonctionnalités - Tests de rétrocompatibilité et migration

---

Mise en contexte

Déprécier une fonctionnalité, c'est annoncer qu'elle sera retirée dans une version future. C'est une décision stratégique qui concerne directement la qualité : les

utilisateurs et les systèmes qui dépendent de cette fonctionnalité doivent avoir le temps et les outils pour migrer vers l'alternative. Un processus de dépréciation mal géré est une source majeure de défauts, de plaintes et de perte de confiance dans le produit.

Processus de dépréciation géré en qualité

7. **Annonce avec préavis suffisant :** Communiquer la dépréciation avec un délai raisonnable (généralement au moins une version MAJEUR d'avance).
8. **Documentation de l'alternative :** Fournir une alternative claire et documentée à la fonctionnalité dépréciée.
9. **Période de coexistence :** Maintenir la fonctionnalité dépréciée ET son alternative pendant la période de migration, toutes deux testées.
10. **Tests de rétrocompatibilité :** Valider que les utilisateurs existants ne sont pas impactés pendant la période de coexistence.
11. **Tests de migration :** Valider que la migration vers l'alternative fonctionne correctement pour tous les cas d'usage documentés.
12. **Retrait et validation finale :** Retirer la fonctionnalité et valider qu'aucune référence ne subsiste dans les systèmes internes.

Tests spécifiques à la gestion de la dépréciation

Type de test	Objectif	Quand l'exécuter
Tests de coexistence	Vérifier que la fonctionnalité dépréciée et son alternative fonctionnent correctement en parallèle	Dès l'annonce de la dépréciation jusqu'au retrait
Tests de migration	Vérifier que la migration des données, configurations et intégrations vers l'alternative est correcte et complète	Pendant la période de migration
Tests de rétrocompatibilité	Vérifier que les clients existants utilisant l'ancienne fonctionnalité ne sont pas impactés prématurément	À chaque release pendant la période de dépréciation

Type de test	Objectif	Quand l'exécuter
Tests de nettoyage	Vérifier après le retrait qu'aucune référence à la fonctionnalité retirée ne subsiste et ne cause d'erreur	Après le retrait officiel

Questions de réflexion - Jour 5 (Semaine 6)

1. As-tu déjà vécu une dépréciation de fonctionnalité mal gérée du point de vue de la qualité? Quelles en étaient les conséquences pour les utilisateurs et pour l'équipe?

2. Comment identifies-tu tous les "consommateurs" d'une fonctionnalité que tu envisages de déprécier, notamment les intégrations internes non documentées?

3. Quel est le délai de préavis raisonnable pour déprécier une fonctionnalité dans ton contexte? Qu'est-ce qui influence cette décision (type d'utilisateurs, criticité, alternatives disponibles)?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM
| TTLST~

JOUR 6

End of Life (EOL) logiciel - Planification qualité de fin de vie

---

Mise en contexte

La fin de vie (End of Life - EOL) d'un logiciel est le moment où l'éditeur cesse officiellement de le maintenir : plus de correctifs de sécurité, plus de support technique, plus de mises à jour. Pour les utilisateurs qui n'ont pas migré, c'est une situation de risque majeur. Pour l'équipe qualité, la planification de la fin de vie est un exercice de gestion des risques qui commence bien avant la date de l'EOL.

Phases de planification qualité EOL

Phase	Horizon	Actions qualité
Préparation	12 à 24 mois avant EOL	Identifier tous les systèmes et intégrations dépendants, documenter les cas d'usage critiques, définir la stratégie de migration ou de remplacement
Planification des tests de migration	6 à 12 mois avant EOL	Créer un plan de test de migration complet, identifier les cas de données critiques, définir les critères de succès de la migration
Tests de migration par vagues	3 à 6 mois avant EOL	Exécuter la migration par groupes d'utilisateurs, valider chaque vague avant la suivante, documenter les problèmes et les solutions
Validation post-migration	Après chaque vague	Tests de régression sur le système cible, validation de la complétude des données migrées, tests de performance sur le nouveau système
Décommissionnement	À la date EOL et après	Archivage des données historiques, validation de l'absence de référence au système retiré, documentation de clôture

Point de vigilance - Risques spécifiques à l'EOL

Les systèmes en EOL représentent des risques de sécurité croissants car ils ne reçoivent plus de correctifs pour les nouvelles vulnérabilités découvertes. Si un produit en EOL reste en production (situation malheureusement fréquente), la surveillance de sécurité et les tests de pénétration doivent être intensifiés, pas réduits.

Questions de réflexion - Jour 6 (Semaine 6)

1. Dans ton organisation, existe-t-il un processus formel de gestion des EOL logiciels? Qui en est responsable, et quelle est la contribution de l'équipe qualité?

2. Comment testes-tu la complétude d'une migration de données lors d'un remplacement de système? Quelles techniques utilises-tu pour valider qu'aucune donnée n'a été perdue ou corrompue?

3. Qu'est-ce qu'un "EOL invisible" - une situation où un composant tiers utilisé par ton produit atteint son EOL sans que l'équipe en soit immédiatement consciente? Comment te prémunir contre ce risque?

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM | TTLST~

JOUR 7

Bilan qualité du cycle de vie complet - Créer ton tableau de bord produit

---

Mise en contexte

Tu arrives au septième jour de la semaine 6 - et à la conclusion de la Partie B. Ce dernier exercice est une synthèse active : créer ton tableau de bord de qualité produit. Ce n'est pas un exercice théorique. C'est l'outil qui va te permettre de piloter la qualité de ton produit sur l'ensemble de son cycle de vie, de communiquer avec tes stakeholders et de prendre des décisions éclairées.

Structure du tableau de bord qualité produit

Dimension	Indicateurs clés (KQI)	Fréquence de mise à jour	Public cible
Qualité des	Taux de défauts par	Par release	Équipe de

Dimension	Indicateurs clés (KQI)	Fréquence de mise à jour	Public cible
livraisons	release, taux d'échappement (défauts trouvés en production), temps moyen de correction		développement, product owner
Couverture de tests	Couverture de code automatisée, pourcentage de cas de test exécutés, taux de réussite des tests automatisés	Continue (par build CI)	Équipe technique, chef de projet
Performance et fiabilité	Temps de réponse moyen et P95/P99, taux de disponibilité, nombre d'incidents en production	Continue (monitoring)	Équipe opérations, stakeholders business
Sécurité	Nombre de vulnérabilités ouvertes par sévérité, délai moyen de correction des CVE critiques	Hebdomadaire ou par scan automatique	Équipe sécurité, direction
Dette technique	Score de qualité de code (SonarQube), complexité cyclomatique moyenne, ratio dette/effort de livraison	Par sprint ou par release	Équipe de développement, chef de projet
Satisfaction utilisateur	NPS (Net Promoter Score), CSAT, taux de tickets de support liés à la qualité	Mensuelle ou par release majeure	Direction, équipe commerciale, product owner

Exercice pratique - Jour 7 (Semaine 6)

Crée la première version de ton tableau de bord qualité produit :

- Sélectionne 5 à 8 indicateurs pertinents pour ton produit et ta phase de

cycle de vie.

- Pour chaque indicateur, définis la valeur actuelle, la valeur cible et le seuil d'alerte.
- Identifie les sources de données pour chaque indicateur (outils existants, processus manuels).
- Définis la fréquence de mise à jour et les responsables pour chaque indicateur.
- Réfléchis au format de communication : comment vas-tu partager ce tableau de bord avec tes différents stakeholders?

Questions de réflexion - Jour 7 (Semaine 6)

1. Quels indicateurs de qualité manquent actuellement dans ton organisation et qu'une mesure régulière améliorerait concrètement la prise de décision?

2. Comment évites-tu le piège du "tableau de bord cosmétique" - un tableau qui affiche de beaux chiffres mais ne reflète pas la vraie qualité perçue par les utilisateurs?

3. Comment ton tableau de bord qualité doit-il évoluer quand le produit passe d'une phase à l'autre de son cycle de vie?

TTLSTˆ˜ - Exercice de Totalisation - Semaine 6

Dimension 1 - Substantif linguistique : "Action d'additionner ; action de considérer l'ensemble, le total, de synthétiser."

Dimension 2 - Philosophie ORIGINALIUM : "Rassembler et réaliser

totalement dans son ensemble - avant, pendant et après - en continu, sans interruption."

Exercice de totalisation : La semaine 6 t'a invité à voir la qualité dans la durée - non pas comme un sprint vers une cible fixe, mais comme un marathon d'adaptation continue. Le cycle de vie d'un produit logiciel est long, changeant, parfois imprévisible. Ta stratégie de qualité doit avoir la même agilité.

Dans tes notes, réponds à ces questions de totalisation :

- **Le cycle entier :** Si tu traçais une ligne du temps de la qualité de ton produit depuis son lancement jusqu'à aujourd'hui (ou jusqu'à sa fin de vie anticipée), à quoi ressemblerait-elle? Quels ont été les moments de progression, de régression, de stabilité?
- **La qualité comme investissement :** Comment changes-tu ta vision de la qualité en la voyant comme un investissement sur le cycle de vie complet, plutôt que comme un coût par sprint? Quel impact cela a-t-il sur tes conversations avec tes stakeholders?
- **L'après :** Qu'est-ce qu'un produit logiciel laisse comme héritage qualité lorsqu'il atteint sa fin de vie? Comment ce patrimoine est-il préservé et transmis à l'équipe suivante ou au produit successeur?
- **Ta vision continue :** Formule en une phrase ta définition personnelle de ce que signifie assurer la qualité d'un produit logiciel sur l'ensemble de son cycle de vie. C'est ta déclaration de qualité continue.

Totaliser le cycle de vie, c'est comprendre que chaque défaut corrigé aujourd'hui est un cadeau fait aux utilisateurs de demain. Et que chaque décision de qualité prise maintenant est une dette ou un investissement pour l'équipe qui prendra la relève.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Pont vers la Partie C - Ce qui t'attend

Tu as parcouru un territoire exigeant et riche au cours de ces trois semaines. Tu sais maintenant lire un produit logiciel selon son type, dialoguer avec son architecture et accompagner sa qualité à travers les phases de son cycle de vie. C'est une maîtrise que peu de professionnels de la qualité atteignent avec cette profondeur.

La Partie C - les semaines 7 et 8 - va t'emmener vers la Grande Totalisation de L2. Tu vas synthétiser l'ensemble de ce que tu as appris sur le produit logiciel - des fondations de la Partie A aux profondeurs techniques de la Partie B - en une vision unifiée et personnelle de la qualité logicielle. Et tu construiras le pont vers L3 : le cycle de vie du développement logiciel (SDLC), là où le produit prend naissance, grandit et se transforme sous tes yeux - et sous ta vigilance.

Tu es prêt pour ce qui vient. Continue.

Léo Michel Bélanger - BEASTEST ORIGINALIUM

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Notes personnelles - Page 1

Tes réflexions, tes insights, tes questions - note tout ici.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Notes personnelles - Page 2

Tes idées d'amélioration, tes constats terrain, tes projets de mise en pratique.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM

Tes totalisations personnelles - ce que tu emportes de la Partie B.

© Léo Michel Bélanger - Tous droits réservés / Images et couverture par Soléane Bélanger / Publié par ORIGINALIUM
| TTLST~